

Laporan UAS Sistem Multimedia Equalizer MATLAB

Identitas Praktikan

Nama: Ariski Myardi

NIM: 1103220019

Kelas: TK-46-G07

Mata Kuliah: Sistem Multimedia

Topik: Aplikasi Equalizer Audio 10-Band berbasis GUI MATLAB

Lingkungan: MATLAB R2026a

1. Pendahuluan

Equalizer audio adalah instrumen pemrosesan sinyal yang bertugas menyesuaikan tingkat penguatan (gain) pada spektrum frekuensi tertentu. Melalui alat ini, karakteristik suara dapat dimodifikasi sesuai preferensi, seperti memperjelas nada bass atau meredam treble. Sasaran utama dari proyek ini adalah mengembangkan perangkat lunak equalizer audio berbasis antarmuka grafis (GUI) menggunakan MATLAB R2026a yang dioperasikan pada platform macOS.

Aplikasi yang dirancang ini memiliki berbagai fungsionalitas: (a) kemampuan memuat file audio sekaligus menampilkan bentuk gelombangnya; (b) modifikasi gain secara terpisah pada 10 pita (band) frekuensi berbeda menggunakan slider; (c) implementasi empat macam filter, yakni Low Pass Filter (LPF), High Pass Filter (HPF), Band Pass Filter (BPF), dan Band Stop Filter (BSF); (d) visualisasi dari respons filter yang sedang aktif; serta (e) fitur pemutaran audio yang dilengkapi dengan kontrol operasional seperti Play, Pause, Resume, Stop, dan pengatur volume. Pembuatan antarmuka secara keseluruhan memanfaatkan elemen uifigure untuk menjamin kompatibilitas dengan MATLAB versi terkini tanpa harus mengandalkan fitur GUIDE yang sudah usang.

2. Tinjauan Pustaka

2.1 Pita Frekuensi dan Penguatan Sinyal (dB)

Spektrum suara dipecah menjadi beberapa pita frekuensi (band). Tiap pita tersebut diberikan nilai penguatan dalam desibel (dB). Konversi dari nilai dB ke dalam bentuk rasio linear menggunakan persamaan $G = 10^{(dB/20)}$. Nilai 0 dB mengindikasikan bahwa sinyal tidak mengalami perubahan. Sementara itu, nilai positif menunjukkan adanya amplifikasi, dan nilai negatif berarti komponen frekuensi pada pita tersebut direduksi.

2.2 Transformasi Fast Fourier (FFT) untuk Domain Frekuensi

Keseluruhan mekanisme equalizer dan filter pada aplikasi ini dieksekusi dalam domain frekuensi dengan memanfaatkan Fast Fourier Transform (FFT). Sinyal awal dikonversi ke domain frekuensi, lalu dikalikan dengan masker penguatan atau respons frekuensi, sebelum akhirnya dikonversi kembali ke domain waktu melalui Inverse FFT (IFFT). Metode ini diimplementasikan secara independen sehingga program tidak bergantung pada Signal Processing Toolbox, sekaligus mencegah galat fungsi yang tidak terdefinisi.

2.3 Karakteristik Filter

LPF berfungsi meloloskan sinyal di bawah frekuensi batas (cutoff) dan memblokir frekuensi di atasnya. Sebaliknya, HPF menahan frekuensi di bawah cutoff dan meloloskan frekuensi di atasnya. BPF dirancang untuk hanya meneruskan sinyal yang berada di antara dua titik cutoff. Terakhir, BSF memblokir rentang frekuensi yang berada di antara dua titik cutoff dan membiarkan sinyal di luar rentang tersebut lewat.

3. Rincian Pita (Band) Equalizer

Aplikasi memecah spektrum audio ke dalam 10 pita frekuensi yang masing-masingnya dapat dikontrol secara mandiri. Tabel di bawah ini merangkum rentang frekuensi untuk tiap pita.

Pita (Band)	Rentang Frekuensi
1	10 - 200 Hz
2	201 - 400 Hz
3	401 - 800 Hz
4	801 - 1500 Hz
5	1501 - 3 KHz
6	3 - 5 KHz
7	5 - 7 KHz
8	7 - 10 KHz
9	10 - 15 KHz
10	> 15 KHz

4. Penjelasan dan Arsitektur Kode

4.1 Lingkungan Pengembangan

Sistem ditulis dalam sebuah file tunggal bernama `equalizer_gui.m`. File tersebut mencakup fungsi sentral `equalizer_gui` yang bertugas membentuk seluruh GUI dan memuat beberapa nested function sebagai callback. Semua callback mengakses sebuah struktur data (struct) bernama `S` yang menyimpan informasi data audio, sample rate, nilai gain masing-masing pita, status filter aktif, objek audioplayer, serta tingkat volume. Di samping itu, program ini memuat dua local function, yakni `eqMask` dan `filterMask`, yang berfungsi menangani penyusunan masker pemrosesan sinyal di domain frekuensi.

4.2 Bagian-Bagian Antarmuka

Jendela GUI utama (uifigure berjudul "equalizer") dirancang dalam dua bagian. Bagian kiri berisi: informasi identitas (Nama dan NIM); isian alamat file lengkap dengan tombol Browse; modul Equalizer berisi 10 slider vertikal berentang -15 hingga +15 dB dengan kotak indikator numerik; modul Filter dengan tombol LPF, HPF, BPF, dan BSF; serta modul Setting yang memuat kontrol Play, Pause, Resume, Stop, tombol Reset, dan pengatur Volume. Bagian kanan disiapkan untuk menampilkan tiga jenis grafik: gelombang audio asli (Original), plot bentuk respons dari filter yang dipilih, dan gelombang audio keluaran.

4.3 Daftar Fungsi

`UtamaonBrowse`: Memunculkan dialog pencarian berkas, memuat file audio menggunakan `audioread`, serta merekam nilai sample rate. `loadSignal`: Mengonversi format file menjadi mono, melakukan normalisasi amplitudo, menampilkan grafik sinyal awal, lalu memproses audio tersebut. `onSlider / onValBox`: Menjaga sinkronisasi nilai slider dengan kotak nilai, menyimpan tingkat penguatan (dB) tiap pita, lalu memproses ulang audio. `onFilter`: Memilih atau mematikan fungsi filter (LPF/HPF/BPF/BSF) dan memproses kembali audio. `onReset`: Mengembalikan seluruh konfigurasi gain ke angka 0 dB dan mematikan fungsi filter. `onVolume`: Menyesuaikan pengali volume. Apabila audio tengah berputar, maka pemutaran akan diulang dengan menyesuaikan volume baru. `processAudio`: Inti pemrosesan yang meliputi komputasi FFT sinyal, penerapan masker equalizer maupun filter, IFFT, langkah normalisasi, serta pembaruan plot grafik. `eqMask`: Menyusun masker penguatan untuk masing-masing pita (mengonversi dB ke bentuk linear) yang bersifat simetris terhadap frekuensi Nyquist. `filterMask`: Mengalkulasi respons magnitudo dengan gaya Butterworth orde 6 untuk digunakan pada LPF, HPF, BPF, atau BSF. `plotOriginal / plotFilterShape / plotOutput`: Menggambar visualisasi sinyal asli, profil respons filter, dan sinyal hasil olahan. `onPlay / onPause / onResume / onStop`: Memberikan kendali pemutaran audio memanfaatkan objek audioplayer.

5. Alur Pemrosesan Sinyal

5.1 Mekanisme Equalizer (eqMask)

Fungsi eqMask memproduksi sebuah vektor penguatan di sepanjang bin frekuensi dari hasil FFT. Untuk tiap pitanya, gain dalam format dB dikonversi menjadi rasio linear menggunakan perhitungan $G = 10^{(dB/20)}$ dan diaplikasikan pada bin yang bersesuaian dengan rentang frekuensi pita tersebut. Pemetaan pita ini dikonfigurasi secara simetris terhadap titik frekuensi Nyquist agar keluaran dari langkah IFFT tetap menghasilkan sinyal riil.

5.2 Mekanisme Filter (filterMask)

Fungsi filterMask mendesain kurva respons magnitudo layaknya filter Butterworth orde n = Konfigurasi ini ditujukan supaya kurva memiliki transisi redaman (roll-off) yang halus dan bukan pemotongan secara tegak atau ekstrem. Titik cutoff default untuk setiap jenis filternya adalah sebagai berikut: LPF: Meneruskan sinyal di bawah cutoff ($f_c = 1000$ Hz). HPF: Meneruskan sinyal di atas cutoff ($f_c = 3000$ Hz). BPF: Meneruskan frekuensi di antara dua batas cutoff ($f_1 = 800$ Hz, $f_2 = 4000$ Hz). BSF: Menahan rentang frekuensi antara dua batas cutoff ($f_1 = 1000$ Hz, $f_2 = 3000$ Hz).

5.3 Integrasi Pemrosesan

Kedua masker yang telah dibuat lalu dikombinasikan melalui fungsi processAudio. Gelombang hasil keluaran dihitung lewat algoritma $y = \text{real}(\text{IFFT}(\text{FFT}(x) \times H_{eq} \times H_f))$, yang mana H_{eq} melambangkan masker dari equalizer dan H_f merupakan masker dari filter terpilih. Apabila pengguna tidak menyalakan filter, nilai H_f dipatok menjadi 1 pada seluruh rentang frekuensi, memastikan hanya fungsi equalizer yang memengaruhi audio. Sesudah proses IFFT, sinyal akan melalui tahap normalisasi skala kecil untuk menghindari gejala clipping.

6. Tinjauan Antarmuka dan Visualisasi GUI

Aplikasi menyuguhkan GUI interaktif ketika di-run di atas platform macOS dengan sampel suara bawaan dari MATLAB (handel.mat). Sisi kiri menampilkan panel kontrol secara komprehensif, sementara sisi kanan menyajikan tiga visualisasi grafik yang disusun berurutan ke bawah.

Plot paling atas (berwarna biru) memperlihatkan wujud gelombang awal di domain waktu. Plot bagian tengah (berwarna hijau) memberikan bentuk respons frekuensi $|H(f)|$ dari kombinasi equalizer serta filter yang sedang menyala. Apabila tidak ada filter yang digunakan, plot respons akan berbentuk datar sejajar di angka 1. Pada plot paling bawah (berwarna oranye), diperlihatkan gelombang audio hasil akhir pasca-pemrosesan di domain waktu. Pergeseran nilai pada slider equalizer atau klik pada tombol filter akan secara otomatis memperbarui grafik bagian tengah dan bawah real-time.

7. Panduan Penggunaan dan Uji Coba

Perangkat lunak dapat digunakan dengan terlebih dahulu menyimpan berkas `equalizer_gui.m` lalu mengeksekusi command `equalizer_gui` lewat Command Window di MATLAB. Secara otomatis, aplikasi segera memuat sinyal sampel `handel.mat` sehingga area plot grafik tidak kosong. Anda bisa menekan ikon Browse jika ingin memasukkan file audio sendiri berformat `.wav`. Setelah audio termuat, geser slider di setiap pita frekuensi untuk mengkustomisasi suara, atau klik tombol salah satu filter untuk memantau kurva responsnya, kemudian tekan Play untuk mendengarkan keluaran suaranya. Jika Anda menekan tombol filter yang sama dua kali berturut-turut, maka filter tersebut akan dinonaktifkan, sedangkan pemakaian tombol Reset akan merestorasi seluruh parameter GUI kembali ke setelan pabrik.

8. Penutup

Pengembangan Aplikasi Equalizer Audio 10-band berbekal GUI telah sukses diimplementasikan dan berjalan dengan stabil menggunakan MATLAB R2026a di macOS. Perangkat lunak ini sanggup membuka file audio, menampilkan visualisasi gelombang, mengkustomisasi gain pada 10 pita secara independen, serta menjalankan empat tipe filter (LPF, HPF, BPF, BSF) lengkap dengan visualisasi responsnya. Hasil olah sinyal juga dapat diputar ulang memakai kendali media dan parameter volume yang fungsional. Pemakaian algoritma FFT untuk tulang punggung program menjadikannya lebih ramping, dapat berdiri sendiri, serta sangat cocok difungsikan sebagai bahan edukasi ihwal konsep dasar equalizer dan penapisan frekuensi audio.



<https://youtu.be/waPkG-3YV7A>

```
function equalizer_gui
% EQUALIZER_GUI 10-band audio equalizer + filter (LPF/HPF/BPF/BSF)
% -----
% Nama : Ariski Myardi
% NIM : 1103220019
% Dibuat untuk MATLAB R2026a (macOS) menggunakan komponen uifigure.
%
% Cara pakai:
% >> equalizer_gui
%
% Catatan teknis:
% - Semua pemrosesan (equalizer & filter) dilakukan di domain frekuensi
%   (FFT) sehingga TIDAK membutuhkan Signal Processing Toolbox.
% - 10 band equalizer mengikuti rentang pada GUI acuan:
%   1) 0-200 Hz   2) 201-400 Hz   3) 401-800 Hz   4) 801-1500 Hz
%   5) 1.5-3 KHz  6) 3-5 KHz     7) 5-7 KHz     8) 7-10 KHz
%   9) 10-15 KHz 10) >15 KHz
% - Tombol preset diganti menjadi filter: LPF, HPF, BPF, BSF.
% -----

%% ===== State bersama (dipakai semua nested function) =====
S.fs = 8192; % sampling rate
S.x = []; % audio asli (mono, ternormalisasi, kolom)
S.y = []; % audio hasil proses
S.player = []; % objek audioplayer
S.volume = 1.0; % faktor volume 0..1
```

```

S.filter = 'none'; % filter aktif: none/LPF/HPF/BPF/BSF
S.order = 6; % orde "Butterworth-like" untuk kurva filter

% Tepi 10 band (Hz) -> 11 nilai
S.edges = [0 200 400 800 1500 3000 5000 7000 10000 15000 20000];
S.names = {'0-200Hz','201-400Hz','401-800Hz','801-1500Hz','1.5-3KHz', ...
           '3-5KHz','5-7KHz','7-10KHz','10-15KHz','>15KHz'};
S.gainsDB = zeros(1,10);

% Cutoff default tiap filter (Hz)
S.lpfCut = 1000; % LPF: lolos < 1 kHz
S.hpfCut = 3000; % HPF: lolos > 3 kHz
S.bpfBand = [800 4000]; % BPF: lolos 0.8-4 kHz
S.bsfBand = [1000 3000]; % BSF: tahan 1-3 kHz

%% ===== Jendela utama =====
S.fig = uifigure('Name','equalizer','Position',[80 60 1200 660], ...
               'Color',[0.94 0.94 0.94]);

% --- Header: Nama dan NIM ---
uilabel(S.fig,'Position',[20 612 560 34], ...
        'Text',' Ariski Myardi - 1103220019', ...
        'FontSize',16,'FontWeight','bold','FontColor',[0.85 0.33 0.10], ...
        'BackgroundColor',[1 1 1]);

% --- File Address ---
uilabel(S.fig,'Position',[20 568 95 26],'Text','File Address', ...
        'FontWeight','bold');
S.fileField = uieditfield(S.fig,'text','Position',[118 568 380 28], ...
                          'Value','(belum ada file - klik Browse)','Editable','off');
uibutton(S.fig,'Position',[508 568 70 28],'Text','Browse', ...
         'ButtonPushedFcn','@onBrowse');

%% ===== Panel Equalizer (10 band) =====
S.eqPanel = uipanel(S.fig,'Title','Equalizer','Position',[20 250 560 305], ...
                   'FontWeight','bold');

colW = 54; x0 = 10;
for i = 1:10
    cx = x0 + (i-1)*colW;
    % label band (di atas slider)
    uilabel(S.eqPanel,'Position',[cx-4 252 colW+6 30],'Text',S.names{i}, ...
            'HorizontalAlignment','center','FontSize',9, ...
            'WordWrap','on');
    % slider vertikal -15..+15 dB
    S.slider(i) = uislider(S.eqPanel,'Orientation','vertical', ...
                          'Limits',[-15 15],'Value',0,'MajorTicks',[-15 0 15], ...
                          'MinorTicks',[],'Position',[cx+6 50 3 195], ...
                          'ValueChangingFcn','@(src,ev) onSlider(i,ev.Value), ...
                          'ValueChangedFcn','@(src,ev) onSlider(i,src.Value));
    % kotak nilai (dB)

```

```

        S.valBox(i) = uieditfield(S.eqPanel,'numeric','Position',[cx-2 12 colW-4 24], ...
            'Value',0,'Limits',[-15 15],'ValueDisplayFormat','%0f', ...
            'HorizontalAlignment','center', ...
            'ValueChangedFcn',@(src,ev) onValBox(i,src.Value));
    end

%% ===== Panel Filter (pengganti Presets) =====
S.fPanel = uipanel(S.fig,'Title','Filter','Position',[20 158 560 84], ...
    'FontWeight','bold');
fNames = {'LPF','HPF','BPF','BSF'};
for k = 1:4
    uibutton(S.fPanel,'Position',[15+(k-1)*135 14 120 40], ...
        'Text',fNames{k},'FontWeight','bold','FontSize',13, ...
        'ButtonPushedFcn',@(src,ev) onFilter(fNames{k}));
end

%% ===== Panel Setting (Play/Pause/Resume/Stop/Volume) =====
S.sPanel = uipanel(S.fig,'Title','Setting','Position',[20 20 560 128], ...
    'FontWeight','bold');
uibutton(S.sPanel,'Position',[15 55 90 36],'Text','Play', ...
    'ButtonPushedFcn',@onPlay);
uibutton(S.sPanel,'Position',[112 55 90 36],'Text','Pause', ...
    'ButtonPushedFcn',@onPause);
uibutton(S.sPanel,'Position',[209 55 90 36],'Text','Resume', ...
    'ButtonPushedFcn',@onResume);
uibutton(S.sPanel,'Position',[306 55 90 36],'Text','Stop', ...
    'ButtonPushedFcn',@onStop);
% reset gain + filter
uibutton(S.sPanel,'Position',[15 14 187 30],'Text','Reset Equalizer & Filter', ...
    'ButtonPushedFcn',@onReset);
% volume
uilabel(S.sPanel,'Position',[410 88 130 20],'Text','Volume', ...
    'HorizontalAlignment','center','FontWeight','bold');
S.volSlider = uislider(S.sPanel,'Limits',[0 1],'Value',1, ...
    'MajorTicks',[0 0.5 1],'Position',[412 60 130 3], ...
    'ValueChangedFcn',@(src,ev) onVolume(src.Value));

%% ===== Sumbu (axes) di kanan =====
pAx = uipanel(S.fig,'Title','Axes','Position',[600 20 580 626], ...
    'FontWeight','bold');
S.axOrig = uiaxes(pAx,'Position',[20 410 540 185]);
title(S.axOrig,'Original'); grid(S.axOrig,'on');

S.axFilt = uiaxes(pAx,'Position',[20 210 540 185]);
title(S.axFilt,'Filter (bentuk respons yang dipilih)'); grid(S.axFilt,'on');

S.axOut = uiaxes(pAx,'Position',[20 10 540 185]);
title(S.axOut,'Hasil audio setelah difilter'); grid(S.axOut,'on');

%% ===== Demo awal: sinyal 'handel' bawaan MATLAB =====
try

```

```

d = load('handel.mat');      % menyediakan y dan Fs
S.fs = d.Fs;
loadSignal(d.y, 'Demo bawaan MATLAB: handel.mat');
catch
    % jika gagal, biarkan kosong sampai user Browse
end

%%%

```

```

%%%          NESTED FUNCTIONS
%%%

```

```

function onBrowse(~,~)
    [f,p] = uigetfile({'*.wav;*.mp3;*.flac;*.ogg;*.m4a','Audio Files'; ...
        '*..*','All Files'},'Pilih file audio');
    if isequal(f,0); return; end
    full = fullfile(p,f);
    try
        [data,fs] = audioread(full);
        S.fs = fs;
        loadSignal(data, full);
    catch ME
        uialert(S.fig,sprintf('Gagal membaca file:\n%s',ME.message),'Error');
    end
end

```

```

function loadSignal(data, label)
    data = mean(double(data),2);      % jadikan mono
    m = max(abs(data));
    if m > 0, data = data/m; end      % normalisasi
    S.x = data(:);
    S.fileField.Value = label;
    plotOriginal();
    processAudio();
end

```

```

function onSlider(idx, val)
    S.gainsDB(idx) = val;
    S.valBox(idx).Value = round(val);
    processAudio();
end

```

```

function onValBox(idx, val)
    val = max(min(val,15),-15);
    S.gainsDB(idx) = val;
    S.slider(idx).Value = val;
    processAudio();
end

```

```

function onFilter(type)

```

```

        if strcmp(S.filter,type)
            S.filter = 'none';          % klik lagi = matikan filter
        else
            S.filter = type;
        end
        processAudio();
    end

function onReset(~,~)
    S.gainsDB = zeros(1,10);
    S.filter = 'none';
    for i = 1:10
        S.slider(i).Value = 0;
        S.valBox(i).Value = 0;
    end
    processAudio();
end

function onVolume(v)
    S.volume = v;
    % jika sedang play, terapkan langsung dengan memutar ulang
    if ~isempty(S.player) && isValid(S.player) && isplaying(S.player)
        onPlay();
    end
end

%% --- Pemrosesan inti ---
function processAudio()
    if isempty(S.x), return; end
    N = numel(S.x);
    f = (0:N-1)'/N*S.fs;          % frekuensi tiap bin
    % 1) Equalizer (gain per band)
    Heq = eqMask(f, S.fs, S.gainsDB, S.edges);
    Xeq = fft(S.x) .* Heq;
    % 2) Filter terpilih
    Hf = filterMask(f, S.fs, S.filter, S);
    S.y = real(ifft(Xeq .* Hf));
    % normalisasi ringan agar tidak clipping
    m = max(abs(S.y));
    if m > 1, S.y = S.y/m; end
    plotFilterShape(f, Heq.*Hf);
    plotOutput();
end

function plotOriginal()
    if isempty(S.x), return; end
    t = (0: numel(S.x)-1)/S.fs;
    plot(S.axOrig, t, S.x, 'Color',[0 0.45 0.74]);
    title(S.axOrig,'Original');
    xlabel(S.axOrig,'Time (s)'); ylabel(S.axOrig,'Amplitude');
    xlim(S.axOrig,[0 t(end)]); ylim(S.axOrig,[-1.05 1.05]);

```

```

        grid(S.axOrig,'on');
    end

    function plotOutput()
        if isempty(S.y), return; end
        t = (0:numel(S.y)-1)/S.fs;
        plot(S.axOut, t, S.y, 'Color',[0.85 0.33 0.10]);
        title(S.axOut,'Hasil audio setelah difilter');
        xlabel(S.axOut,'Time (s)'); ylabel(S.axOut,'Amplitude');
        xlim(S.axOut,[0 t(end)]); ylim(S.axOut,[-1.05 1.05]);
        grid(S.axOut,'on');
    end

    function plotFilterShape(f, H)
        % tampilkan magnitudo gabungan EQ+filter pada 0..Nyquist
        half = f <= S.fs/2;
        ff = f(half);
        mag = abs(H(half));
        plot(S.axFilt, ff, mag, 'Color',[0.20 0.55 0.20],'LineWidth',1.5);
        if strcmp(S.filter,'none')
            ttl = 'Filter: (tidak ada) - hanya equalizer';
        else
            ttl = sprintf('Filter terpilih: %s', S.filter);
        end
        title(S.axFilt, ttl);
        xlabel(S.axFilt,'Frequency (Hz)'); ylabel(S.axFilt,'|H(f)|');
        xlim(S.axFilt,[0 S.fs/2]); ylim(S.axFilt,[0 max(1.2,max(mag)*1.1)]);
        grid(S.axFilt,'on');
    end

    %% --- Pemutaran audio ---
    function onPlay(~,~)
        if isempty(S.y)
            uialert(S.fig,'Belum ada audio. Klik Browse dulu.','Info'); return;
        end
        if ~isempty(S.player) && isvalid(S.player), stop(S.player); end
        out = S.y * S.volume;
        out = max(min(out,1),-1); % cegah clipping
        S.player = audioplayer(out, S.fs);
        play(S.player);
    end

    function onPause(~,~)
        if ~isempty(S.player) && isvalid(S.player), pause(S.player); end
    end

    function onResume(~,~)
        if ~isempty(S.player) && isvalid(S.player), resume(S.player); end
    end

    function onStop(~,~)

```

```

        if ~isempty(S.player) && isvalid(S.player), stop(S.player); end
    end

```

```

end % ===== akhir equalizer_gui =====

```

```

%%

```

```

%%          FUNGSI BANTU (local functions)
%%

```

```

function H = eqMask(f, fs, gainsDB, edges)
% Bangun masker gain (linear) per band, simetris terhadap Nyquist.
fm = f; idx = f > fs/2; fm(idx) = fs - f(idx); % cermin frekuensi
H = ones(numel(f),1);
for b = 1:numel(gainsDB)
    g = 10^(gainsDB(b)/20);          % dB -> linear
    lo = edges(b); hi = edges(b+1);
    sel = (fm >= lo) & (fm < hi);
    H(sel) = g;
end
end

function H = filterMask(f, fs, type, S)
% Respons magnitudo "Butterworth-like" (orde n) di domain frekuensi.
n = S.order;
fm = f; idx = f > fs/2; fm(idx) = fs - f(idx);
fm = max(fm, 1e-6);
switch type
    case 'LPF'
        fc = S.lpfCut;
        H = 1 ./ sqrt(1 + (fm./fc).^(2*n));
    case 'HPF'
        fc = S.hpfCut;
        H = 1 ./ sqrt(1 + (fc./fm).^(2*n));
    case 'BPF'
        f1 = S.bpfBand(1); f2 = S.bpfBand(2);
        Hh = 1 ./ sqrt(1 + (f1./fm).^(2*n));
        Hl = 1 ./ sqrt(1 + (fm./f2).^(2*n));
        H = Hh .* Hl;
    case 'BSF'
        f1 = S.bsfBand(1); f2 = S.bsfBand(2);
        Hl = 1 ./ sqrt(1 + (fm./f1).^(2*n)); % lolos di bawah f1
        Hh = 1 ./ sqrt(1 + (f2./fm).^(2*n)); % lolos di atas f2
        H = min(Hl + Hh, 1);
    otherwise % 'none'
        H = ones(numel(f),1);
end
H = H(:);
end

```